

and run *pvccreate* on that, but that isn't considered very safe.)

Now, create a physical volume on */dev/sdb1*:

```
$: sudo pvccreate /dev/sdb1
```

If you've got an SSD, I've already told you what to do. Now comes the interesting part. We won't create a new volume group; instead, we'll tell the old one to use the space available in the new physical volume, like shown below:

```
$: sudo vgextend VolGroup /dev/sdb1
```

It's that simple. You can now go ahead and grow your volumes as you need.

LVM and migration

LVM has even more magic. It allows migration with zero downtime.

Let's start with the simplest of cases. If your hard disk has just started making clicking noises and you want to retire it before it crashes for good, since you're using LVM, you don't have to worry.

Let's say */dev/sdb1* is failing. If you have enough space on the rest of the physical volumes in the volume group, your life is very easy. All you need to do is use the following command:

```
$: sudo pvmove /dev/sdb1
```

And all the data from */dev/sdb1* will be redistributed amongst the other physical volumes that constitute your volume group. Now you can just kick the physical volume out of your volume group, as follows:

```
$: sudo vgreduce VolGroup /dev/sdb1
```

But what if you didn't have the luck of having free space in your volume group? Well, one way of addressing this would be to buy a new disk, add it to your volume group, then run *pvmove* and the rest of the commands.

You could also move the contents of one physical volume to another physical volume in the same volume group. Start by creating a single partition on the disk, giving it a type ID 8e00, and running *pvccreate* on it. Let's say the new device node is */dev/sdc1*.

Now run:

```
$: sudo vgextend VolGroup /dev/sdc1
$: sudo pvmove /dev/sdb1 /dev/sdc1
```

You can now remove */dev/sdb1* from your volume group.

Remember that *pvmove* moves data, block by block, extent by extent, and while it's generally safe to leave everything mounted and in-use, *pvmove* does take a lot of

time and uses a lot of disk I/O, so disk access is going to be very slow while *pvmove* does its thing. Also, do make sure you have a UPS system, because if the power goes off while you're moving data around, your volume group is pretty much toasted.

LVM snapshots

LVM also allows you to create snapshots of logical volumes, and these are *writable* snapshots. LVM snapshots are 'copy-on-write', which means that when you create a snapshot, it hardly takes up any space because the files in the snapshot are just pointers to the existing data. As you modify data in the snapshot, the pointers for the changed data begin to point to the new files, while the pointers to the unchanged data continue to point to the old files.

To create a snapshot, you need to decide how much write-space you want to dedicate to it. If you give 2 GB of write-space to a snapshot, you'll be able to accumulate 2 GB of changes to your snapshot.

This is how you create a snapshot:

```
$: sudo lvcreate -L 2G -n RootSnapshot --snapshot /dev/
VolGroup/Root
```

The snapshot will be available at */dev/VolGroup/RootSnapshot* and */dev/mapper/VolGroup-RootSnapshot* for you to mount and use.

Encryption

LVM also allows you to create mirrored volumes, and there are some pretty complex migration recipes that allow you to do anything—from distributing specific logical volumes on specific physical volumes to migrating logical volumes between multiple volume groups, and more; but trust me when I say you don't want your head to explode now. LVM is very powerful. You should probably read TLDP's 'LVM-Howto' if you want to get really serious with LVM.

Since LVM exports its volumes as standard Linux block devices, you can do anything you ever wanted to do with standard block devices. You can use them as disks for KVM, or you can create a physical volume inside a logical volume, and create an LVM set inside an LVM set. But one of the more useful things you can do with logical volumes is encryption.

There are two standard encryption recipes that you use with LVM - LVM inside an encrypted block device, or encryption on an LVM logical volume. I'll stick with the latter, since that's the cleaner solution, and the only one you can use if you're spanning across multiple disks.

You can use TrueCrypt or LUKS. I'll talk about LUKS, since it's a bit more flexible.

Let's say you want to encrypt 'Root'. Start out by setting up LUKS, as shown below: